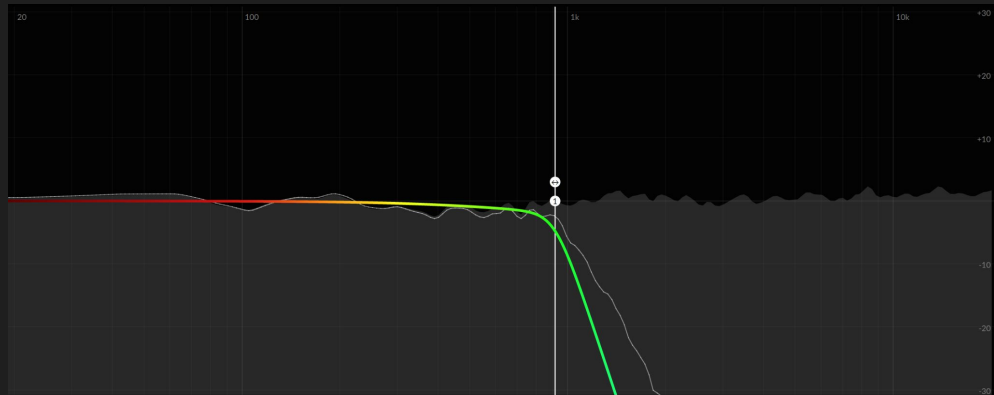


Filter Design

Outline

- What is a filter?
- 1-pole filter derivation
 - Basic circuit modelling
- Extension to 2nd-order SVF
- Filter design challenges
- Filter use cases
- Parameter modulation

What is a filter?

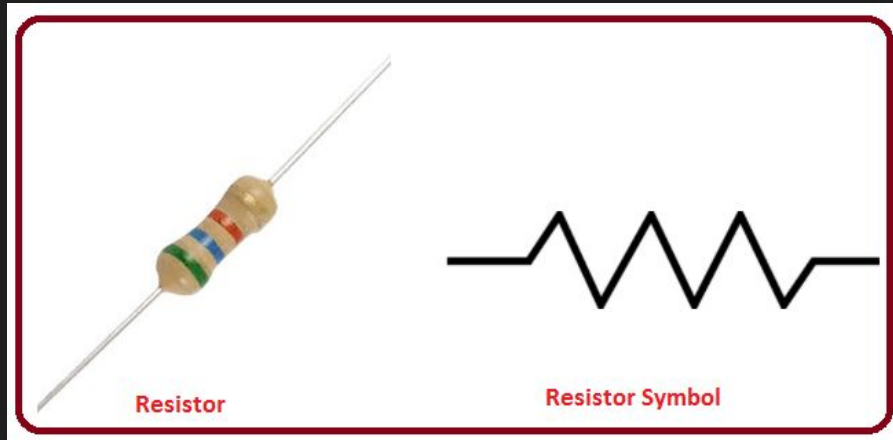


- Mechanism for shaping the frequency spectrum
 - Filter out, boost, or attenuate sound at certain frequencies
 - While leaving sounds at other frequencies untouched
- Analog designs: Sallen-Key, Butterworth
- Digital filters
 - “Optimal” filters: Chebyshev, Elliptic, etc.
 - Adapted from analog designs
 - Digital design with similar properties
 - Analog modelling

Circuits Background

Resistors

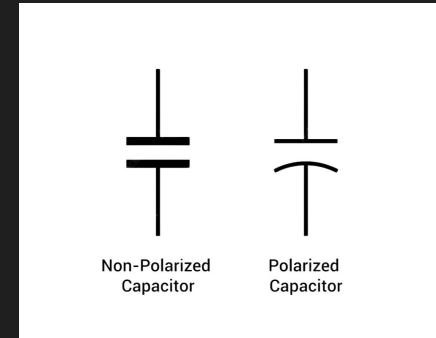
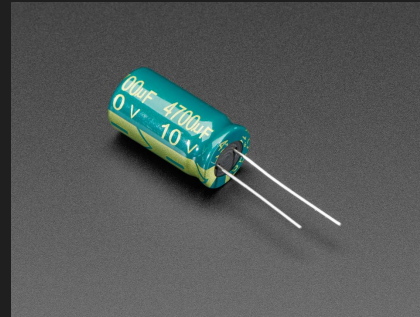
- Ohm's Law: $V = I * R$
- No time dependence
 - Continuous and discrete behavior is the same



Capacitors

$$I(t) = C \frac{dV(t)}{dt}$$

- Has time dependence
- Needs to be discretized



Capacitor Discretization

Discrete time signal (sample rate f_s , sampling period $T = \frac{1}{f_s}$)

$$t = n * T$$
$$x(t) \rightarrow x(n * T) \rightarrow x[n]$$

Discrete time integration: $y(t) = \int x(t)dt$

- Forwards Euler: $y[n] \approx y[n - 1] + Tx[n - 1]$
- Backwards Euler: $y[n] \approx y[n - 1] + Tx[n]$
- Trapezoidal Rule: $y[n] \approx y[n - 1] + \frac{T}{2}(x[n] + x[n - 1])$

Capacitor Discretization

We want: an expression for the current through the capacitor, that we can use for nodal analysis.

Re-write capacitor law as integral:

$$v_c(t) = \frac{1}{C} \int i_c(t) dt$$

Apply trapezoidal rule:

$$v_c[n] = v_c[n-1] + \frac{T}{2C} (i_c[n] + i_c[n-1])$$

Capacitor Discretization

Solve for $i_c[n]$:

$$i_c[n] = \frac{2C}{T}v_c[n] - \frac{2C}{T}v_c[n-1] - i_c[n-1]$$

Define "conductance" constant:

$$g_c = \frac{2C}{T}$$

Define "equivalent current":

$$i_{c_{eq}}[n] = g_c v_c[n] + i_c[n]$$

And simplify:

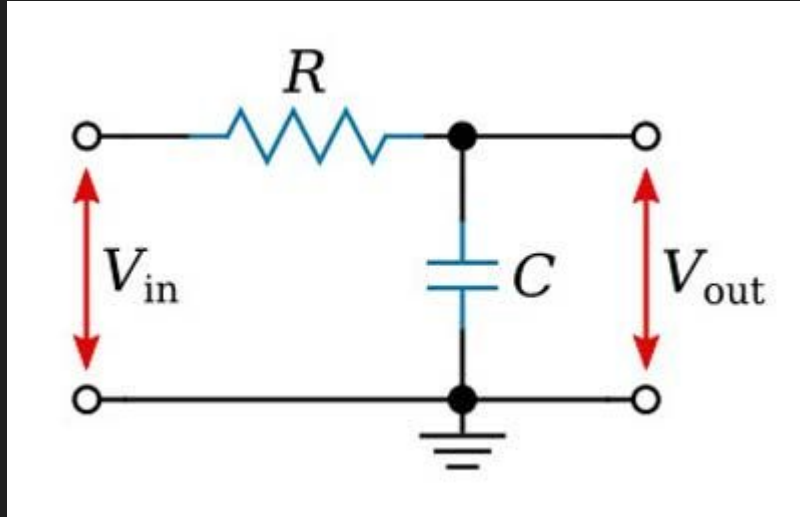
$$i_c[n] = g_c v_c[n] - i_{c_{eq}}[n-1]$$

Capacitor Discretization

i_{ceq} is treated as the capacitor "state", so we need to also derive a "state update" expression.

$$\begin{aligned}i_{ceq}[n] &= g_c v_c[n] + i_c[n] \\i_{ceq}[n] &= g_c v_c[n] + g_c v_c[n] - i_{ceq}[n-1] \\ \boxed{i_{ceq}[n] &= 2g_c v_c[n] - i_{ceq}[n-1]}\end{aligned}$$

Nodal Analysis Model



Simple RC lowpass filter: $i_r[n] = i_c[n]$

Nodal Analysis Model

i_r comes from Ohm's law:

$$i_r[n] = \frac{v_r[n]}{R} = \frac{1}{R}(v_i[n] - v_o[n]) = g_r(v_i[n] - v_o[n])$$

i_c comes from capacitor discretization:

$$i_c[n] = g_c v_c[n] - i_{c_{eq}}[n-1] = g_c(v_o[n] - 0) - i_{c_{eq}}[n-1]$$

Nodal Analysis Model

Solve nodal analysis equation:

$$\begin{aligned}i_r[n] &= i_c[n] \\g_r(v_i[n] - v_o[n]) &= g_c v_o[n] - i_{ceq}[n - 1] \\v_o[n](g_r + g_c) &= g_r v_i[n] + i_{ceq}[n - 1]\end{aligned}$$

$$v_o[n] = \frac{1}{g_r + g_c} (g_r v_i[n] + i_{ceq}[n - 1])$$

State update:

$$i_{ceq}[n] = 2g_c v_o[n] - i_{ceq}[n - 1]$$

Nodal Analysis Model

```
struct State
{
    float iceq {};
};

void process (float* data, int N, float fc, float fs, State& s)
{
    // The system is "over-determined" so we can choose
    // any capacitor value that we want.
    static constexpr auto C = 1.0f;
    const auto R = 1.0f / (2.0f * (float) M_PI * C * fc);
    const auto T = 1.0f / fs;

    const auto g_r = 1.0f / R;
    const auto g_c = 2.0f * C / T;
    const auto g_recip = 1.0f / (g_r + g_c);

    for (int n = 0; n < N; ++n)
    {
        const auto v_i = data[n];

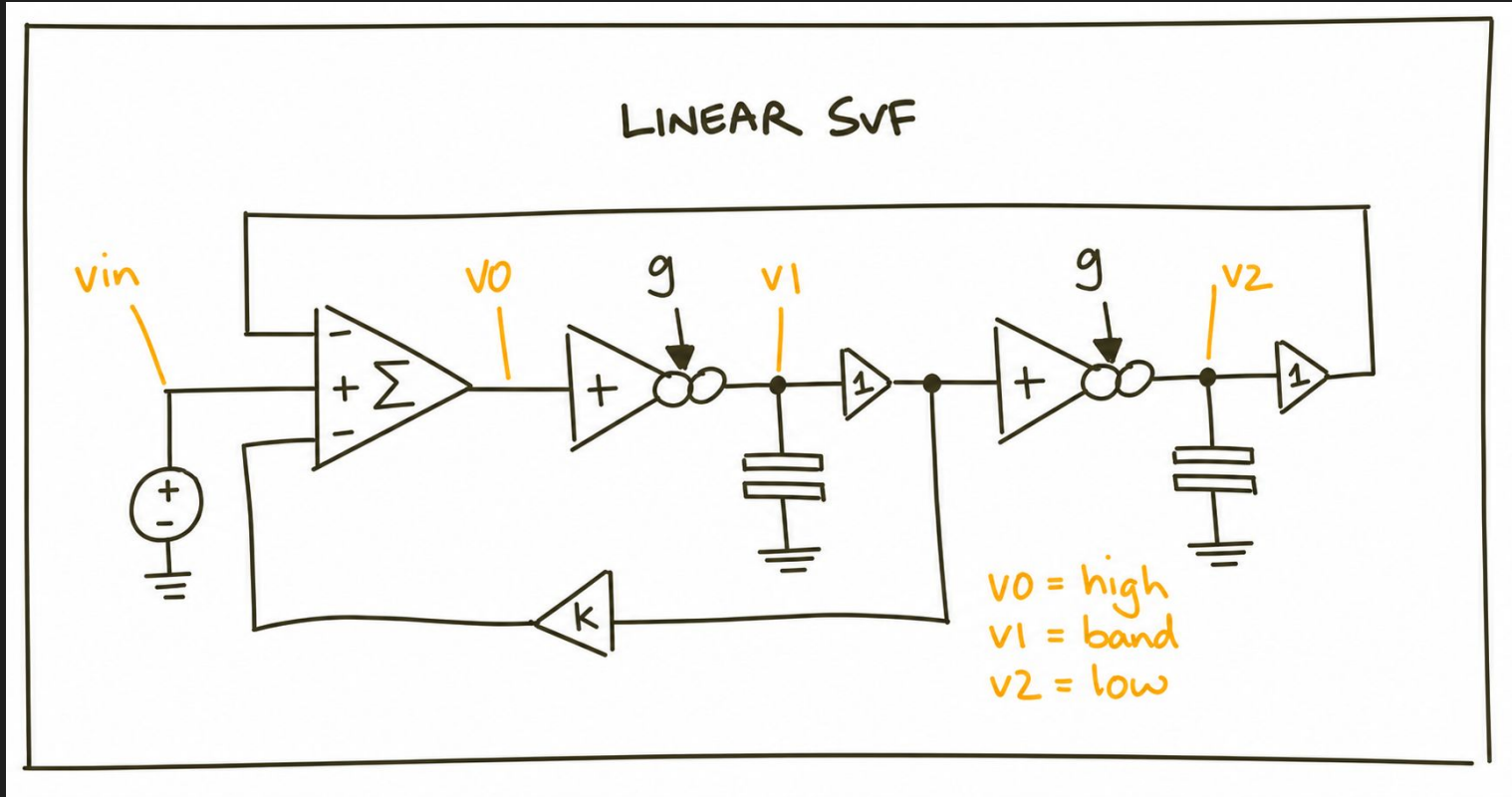
        // compute output
        const auto v_o = g_recip * (g_r * v_i + s.iceq);

        // state update
        s.iceq = 2.0f * g_c * v_o - s.iceq;

        data[n] = v_o;
    }
}
```

- Learning Exercises:
 - Re-derive model using Backwards Euler, instead of trapezoidal integration
 - Derive a model for an RC highpass filter

2nd-order SVF



2nd-order SVF

Nodal analysis equations:

$$v_i[n] - kv_1[n] - v_2[n] = v_0[n]$$

$$gv_0[n] = g_cv_1[n] - i_{c1_{eq}}[n-1]$$

$$gv_1[n] = g_cv_2[n] - i_{c2_{eq}}[n-1]$$

State updates:

$$i_{c1_{eq}}[n] = 2g_cv_1[n] - i_{c1_{eq}}[n-1]$$

$$i_{c2_{eq}}[n] = 2g_cv_2[n] - i_{c2_{eq}}[n-1]$$

Note: $k = 1/Q$

2nd-order SVF

There are multiple ways to solve these nodal analysis equations.

Here's one example:

$$v_0[n] = \frac{1}{(1 + g(g + k))} (v_i[n] - i_{c2_{eq}}[n - 1] - (g + k)i_{c1_{eq}}[n - 1])$$
$$v_1[n] = gv_0[n] + i_{c1_{eq}}[n - 1]$$
$$v_2[n] = gv_1[n] + i_{c2_{eq}}[n - 1]$$

Note: different solutions might be faster on different CPUs
(Reducing total ops vs. maximizing parallelism)

Filter Design Challenges

Frequency Warping

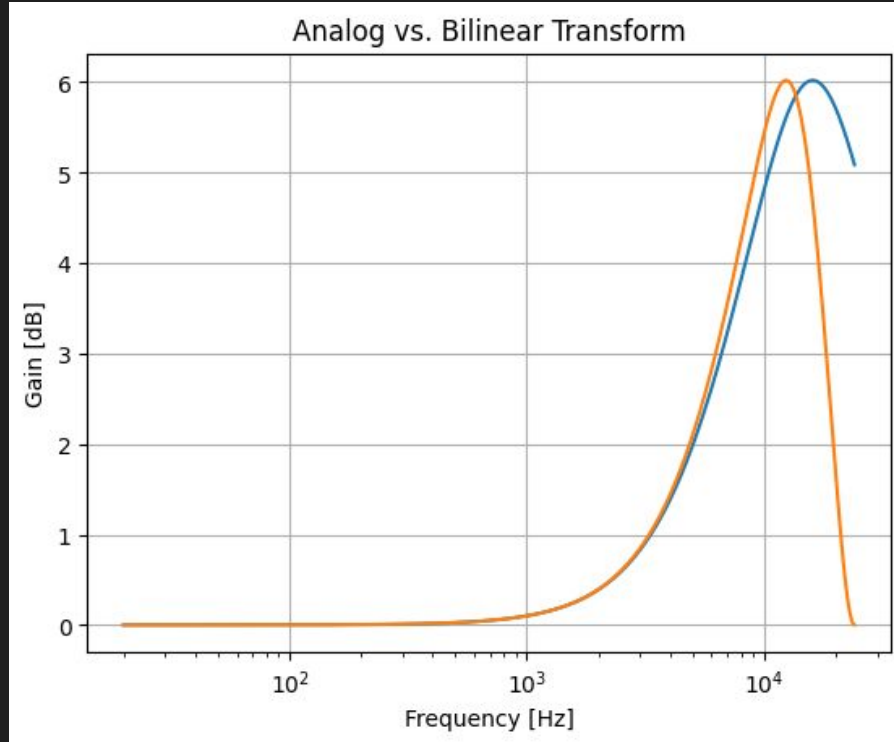
- The trapezoidal rule does not correctly map frequencies close to Nyquist
- Trapezoidal integration can be adjusted to compensate

Cramping

- High frequency features can be “cramped” near Nyquist
- Solutions:
 - Oversampling
 - Modified filter designs (see Vicanek, Ivantsov, etc)

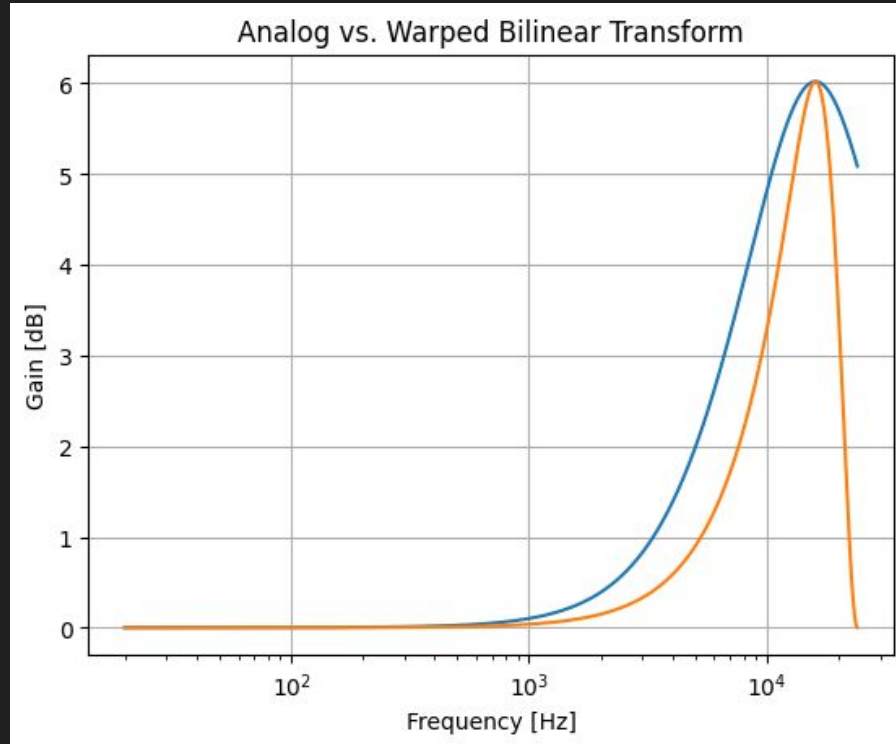
Filter Design Challenges

Warping and Cramping:



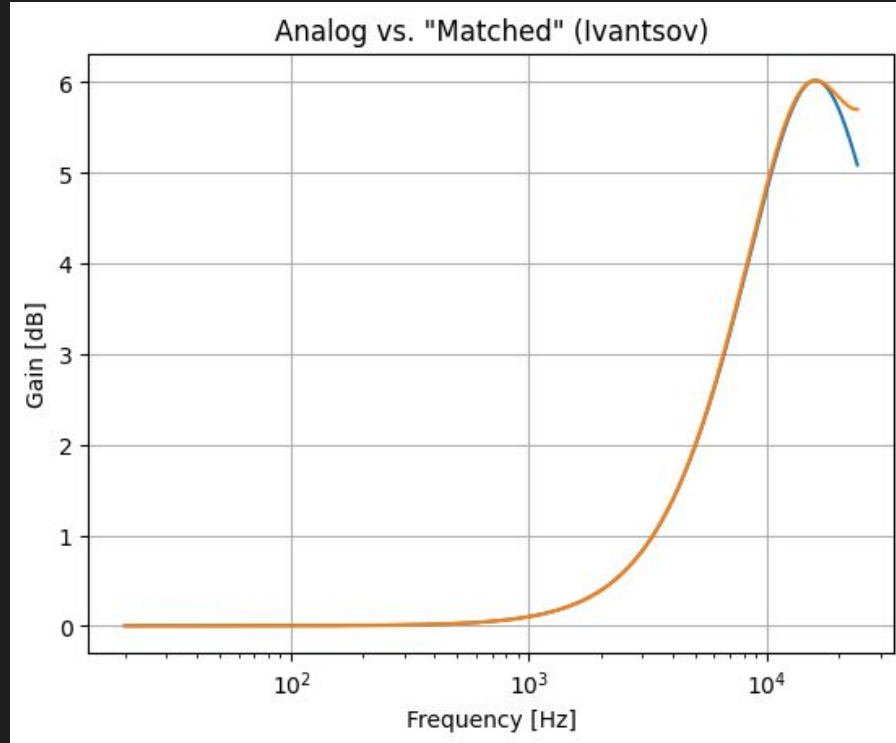
Filter Design Challenges

Modified trapezoidal rule resolves cramping:



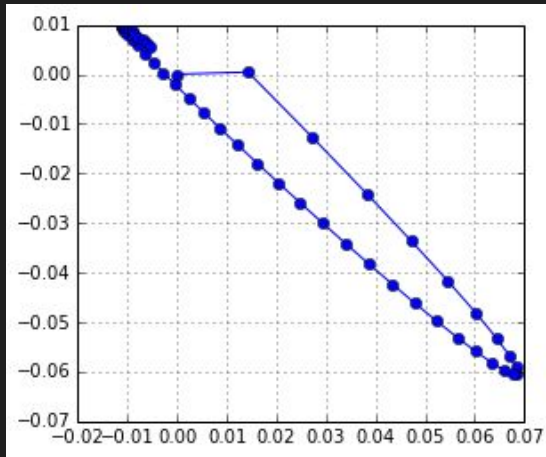
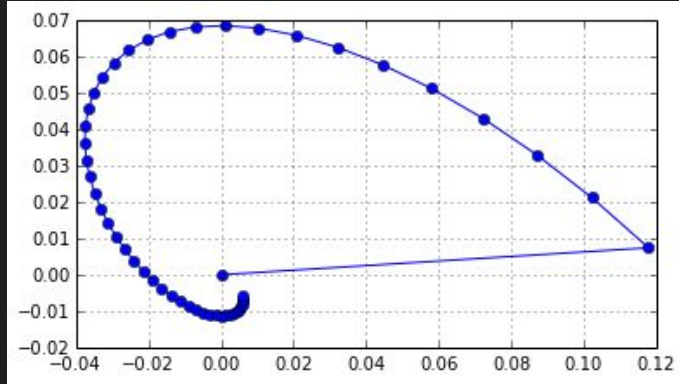
Filter Design Challenges

Ivantsov method resolves cramping:

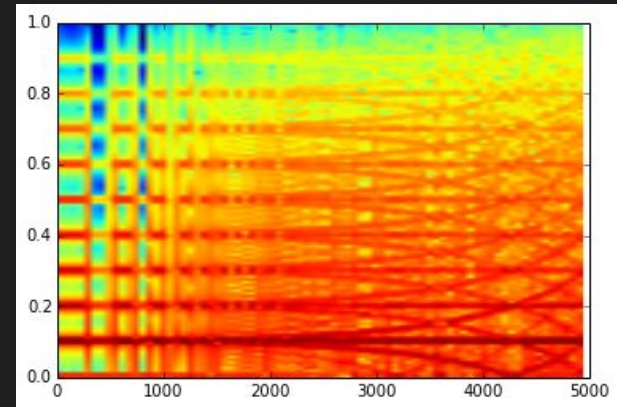
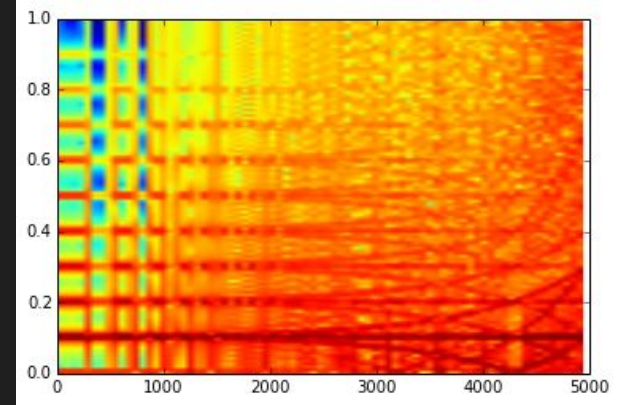


SVF vs. Biquad

More numerically robust



Behaves better under fast modulation



Use Cases



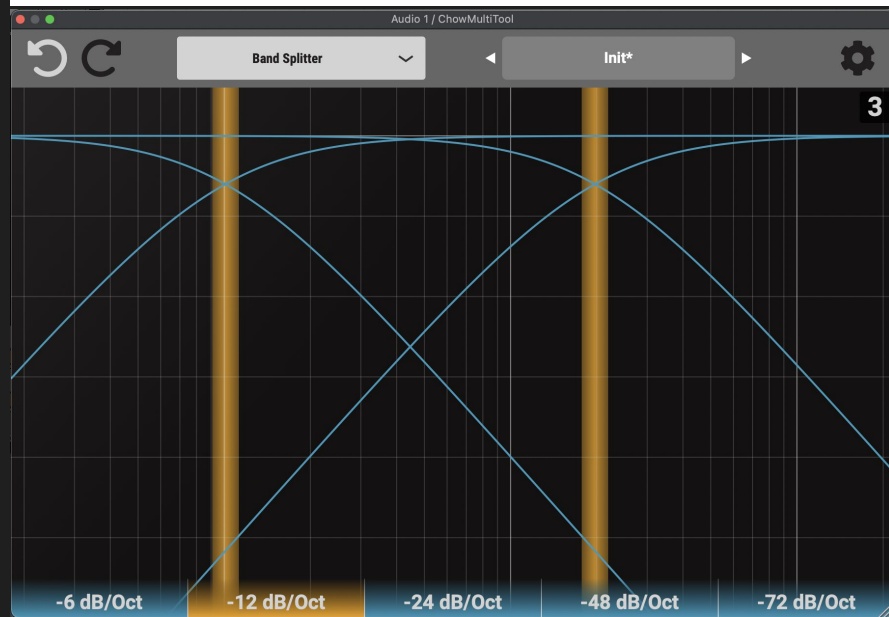
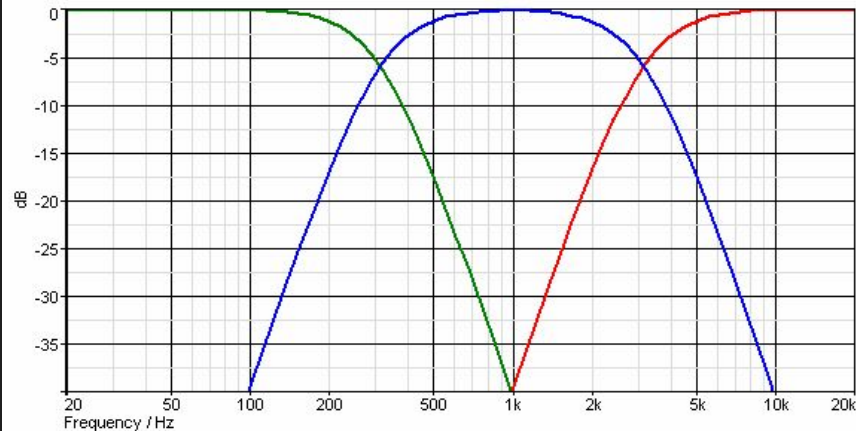
- EQs and mixing effects
- Sound design and creative effects



Use Cases

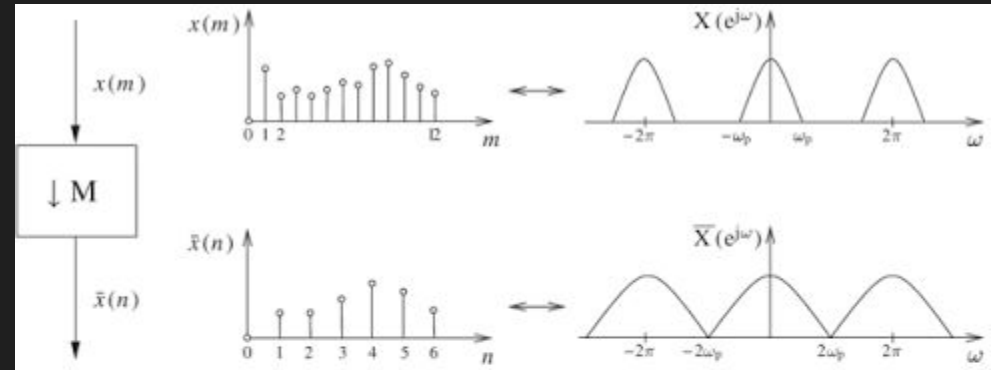
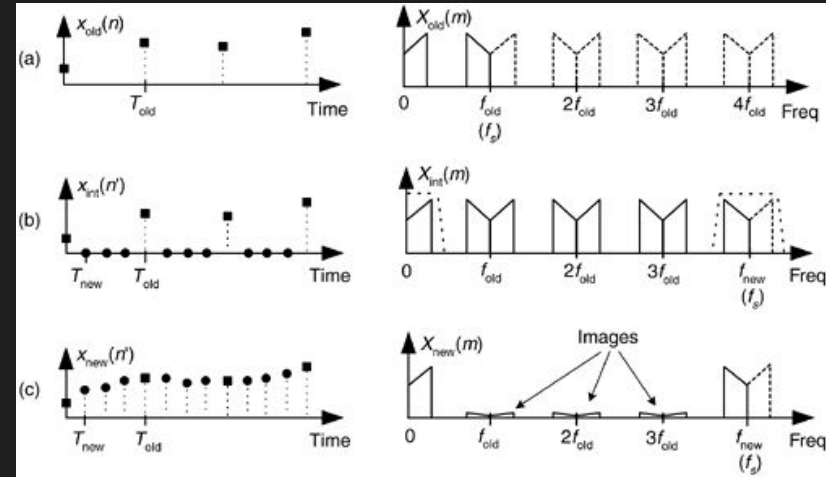
Crossover Filters/Band-Splitters

- Useful for creating multi-band effects
- Crossover point = -6dB
- 2nd-order:
 - LPF/HPF with $K=2$ ($Q = 0.5$)
 - -12dB/octave slope
 - Need to invert phase in one band
- 4th-order:
 - 2x LPF/HPF with $K = \sqrt{2}$ ($Q = 1/\sqrt{2}$)
 - -24dB/octave slope
- Pro tip: 2nd-order SVF has both HPF and LPF outputs...



Anti-Aliasing Filters

- Useful for up-sampling/down-sampling
- High-order lowpass filter
 - Remove all signal above Nyquist frequency
 - Don't affect any signal below Nyquist frequency
 - "Brickwall"
- Upsampling ("interpolation")
 - Zero-stuff → Anti-Imaging Filter
- Downsampling ("decimation")
 - Anti-Aliasing Filter → Decimate



Future References

- Analog modelling
 - Linear circuits can be modelled similarly with nodal analysis
 - Other approaches: Wave Digital Filters, Port-Hamiltonian, etc.
 - Nonlinear circuits
- Filter design references:
 - [Andy Simper \(Cytomic\)](#)
 - [Vicanek de-cramped filter designs](#)
 - [Ivantsov de-cramped filter designs](#)

Wah-Wah Filter

